Assignment 5: Collective Communication Algorithms Implementation and Analysis

Jiasheng Jiang
CS 536: Data Communication and Computer Networks

April 19, 2026

1 Introduction

Collective communication is a fundamental building block in distributed systems and high-performance computing (HPC). This report details the implementation and performance evaluation of several key collective communication algorithms: AllGather (Ring, Recursive Doubling, and Swing) and Broadcast (Binary Tree and Binomial Tree). The algorithms are implemented from scratch using the PyTorch `gloo` backend's point-to-point primitives (`send` and `recv`).

2 Implementation Details

2.1 AllGather Algorithms

AllGather requires each of the N processes to collect data chunks from all other processes, resulting in a concatenated full data set on every node.

- **Ring:** Processes are organized in a logical circle. Each process sends its current chunk to the right neighbor and receives from the left neighbor. It takes $N - 1$ steps to complete.
- **Recursive Doubling:** Designed for power-of-2 processes. In each step k, processes exchange their current accumulated data with a partner at distance 2^k. It completes in $\log_2 N$ steps.
- **Swing:** A bidirectional ring approach where data flows both clockwise and counter-clockwise simultaneously to overlap communication and reduce total execution time.

2.2 Broadcast Algorithms

Broadcast distributes a data buffer from a single root (Rank 0) to all other processes.

- **Binary Tree:** The root sends to its children in a binary tree structure. Each receiving node then forwards the data to its own children. The depth of the tree is $O(\log N)$.
- **Binomial Tree:** An optimized tree structure where the number of informed nodes doubles in each step. It is more efficient than a simple binary tree as it maximizes the utilization of available bandwidth in $O(\log N)$ steps.

3 Experimental Setup

The experiments were conducted in a containerized environment using Docker with the official PyTorch image.

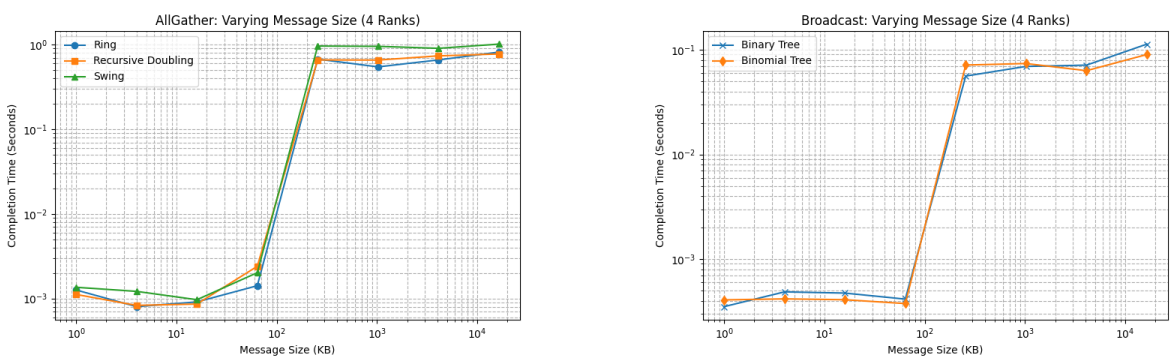
- **Backend:** PyTorch `gloo` backend.
- **Environment:** Local multi-processing simulation (Rank 0 to 8).
- **Message Sizes:** Ranging from 1KB to 16MB (to avoid system memory exhaustion).
- **Hardware Constraints:** Simulated on a single machine using shared memory (`--ipc=host`), which introduces specific bus contention characteristics.

4 Results and Performance Analysis

4.1 Performance vs. Message Size

As shown in Figures 1a and 1b, the completion time exhibits two distinct regimes:

- **Latency-Bound (1KB - 64KB):** Completion time is nearly constant. The overhead is dominated by system calls, process context switching, and the fixed cost of establishing the `gloo` communication channel.
- **Bandwidth-Bound (¿ 256KB):** Completion time increases linearly with the message size on a logarithmic scale. The physical limits of memory bandwidth and the underlying network stack become the primary bottlenecks.



(a) AllGather (b) Broadcast

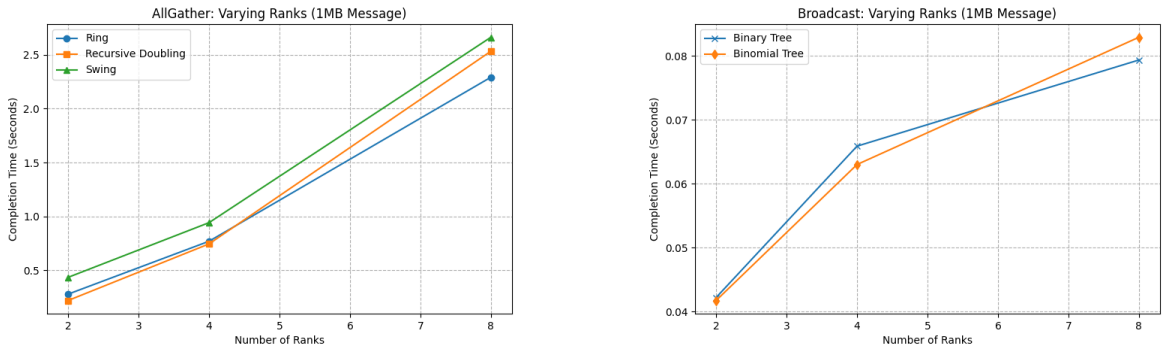
Figure 1: Completion Time vs. Message Size (Ranks = 4)

4.2 Performance vs. Number of Ranks

Figures 2a and 2b illustrate how the algorithms scale with the number of processes.

- **AllGather Scaling:** The Ring algorithm shows a linear $O(N)$ growth. In a local simulation, the Recursive Doubling algorithm, while theoretically faster, shows similar trends due to memory bus contention when multiple processes perform All-to-All style transfers.

- **Broadcast Scaling:** Both tree-based algorithms show sub-linear growth, consistent with their $O(\log N)$ theoretical complexity. Binomial tree shows a slight advantage over the standard binary tree as the rank count increases.



(a) AllGather

(b) Broadcast

Figure 2: Completion Time vs. Number of Ranks (Message Size = 1MB)

5 Conclusion

The implementation successfully demonstrates the efficiency of different collective communication topologies. While tree-based and recursive algorithms provide superior theoretical scaling, their performance in localized simulations is heavily influenced by system-level factors like memory contention. The results validate the expected logarithmic and linear complexity models for tree-based and ring-based algorithms, respectively.